

Machine Intelligence

5. Machine Learning: Supervised Learning and Decision Trees

Learning when Someone Tells You The Right Answers!

Álvaro Torralba



AALBORG UNIVERSITET

Fall 2024

So far

The general pattern so far:

Problem/Domain description	State Space Problem	CSP	Probabilistic Model
Inference Algorithms	Search ($A^*, \dots$)	Arc Consistency, Variable Elimination	Variable Elimination
Solutions	Goal states, plans, diagnoses, predictions,...		

- Problem/Domain description and algorithms designed by human
- **Deduction:** Given our knowledge what conclusions can we get?

Objective of (machine) **learning**:

- Agent can *learn by experience*: improve performance over time
- Agent (program or domain description) can be automatically constructed from *examples* (rather than designed by expert)
- **Induction**: generalize from examples/observations

Inductive reasoning is distinct from deductive reasoning. If the premises are correct, the conclusion of a deductive argument is certain; in contrast, the truth of the conclusion of an inductive argument is probable, based upon the evidence given.[Wikipedia]

Types of Learning

Supervised Learning (Chapters 5-7)

- Learn a function from input/output examples
- Task: predict some (unobserved) **target** or **class** variable based on observed values of (**predictor**) **attributes**
 - **Regression**, if target is continuous
 - **Classification**, if target is discrete
- Model types e.g.: Decision trees, k -nearest neighbors, Neural networks, Naive Bayes, ...

Unsupervised Learning (Chapter 8)

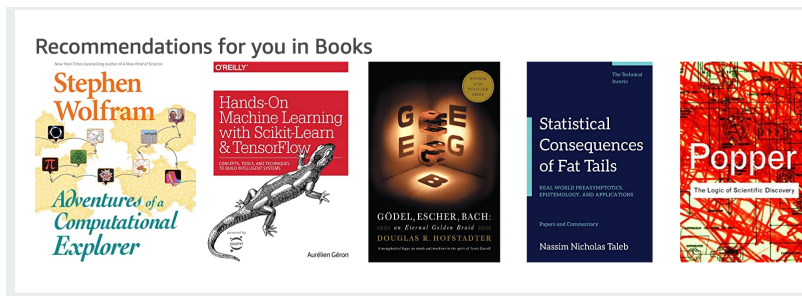
- Learn patterns in the input data
- Task: **Clustering**: identify coherent subgroups in data
- Model types e.g.: k -means, hierarchical clustering, Self-organizing maps, probabilistic clustering, ...

Reinforcement learning (Chapter 13)

- Task: Learn how to take actions in an environment aiming to maximize a cumulative reward
- Models: Markov decision processes, ...

Some Applications of Supervised Learning

Recommended systems



Automatic Translation

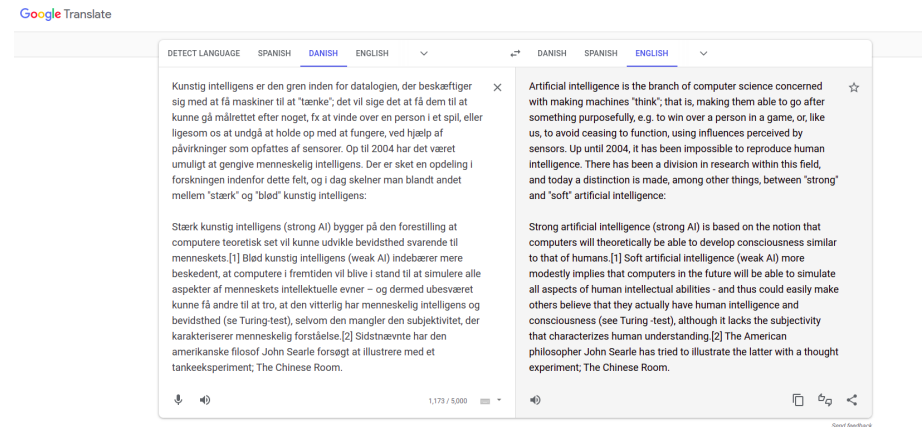
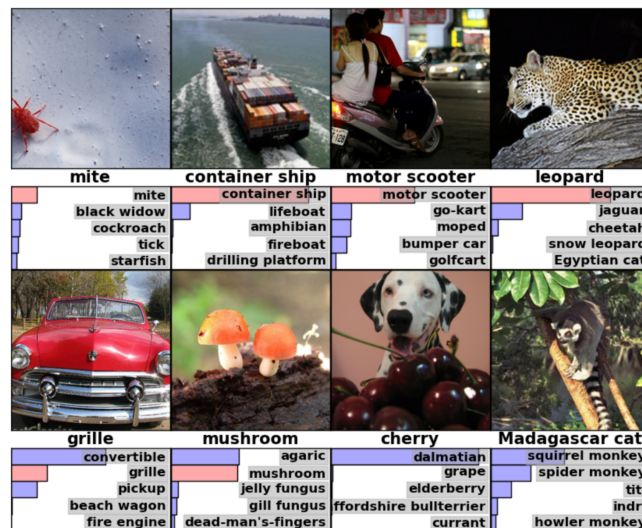
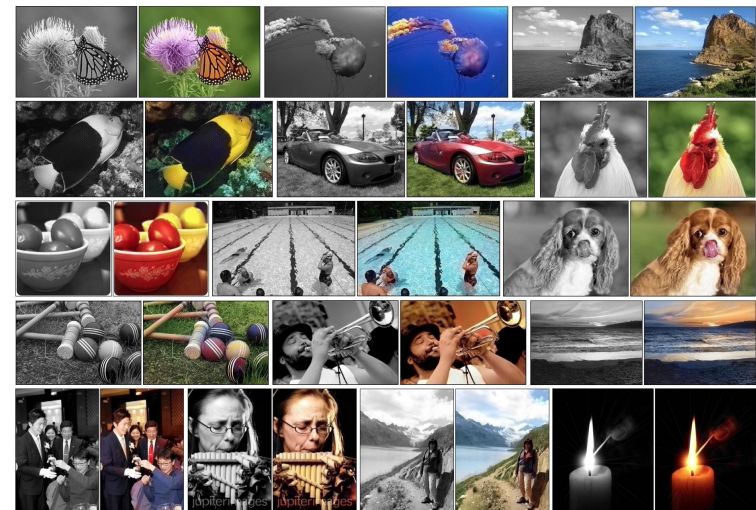


Image Classification



Colorization of images



Our Agenda for This Chapter

- **Supervised Learning:** What is the problem we are trying to solve here?
→ **Basic concepts and definitions**
- **Linear Regression:** Basic regression technique
→ **A basic example of a regression technique to grasp the basic concepts**
- **Decision Trees:** A basic way to structure our knowledge
→ **A basic data structure for classification**
- **Learning Decision Trees:** Algorithm ID3
→ **How learning actually takes place?**

Supervised Learning

In order to show a certain behavior, or perform certain tasks, an agent needs to make the right decisions based on possible observations

Question!

How can the agent (or learning algorithm) know what are the right decisions?

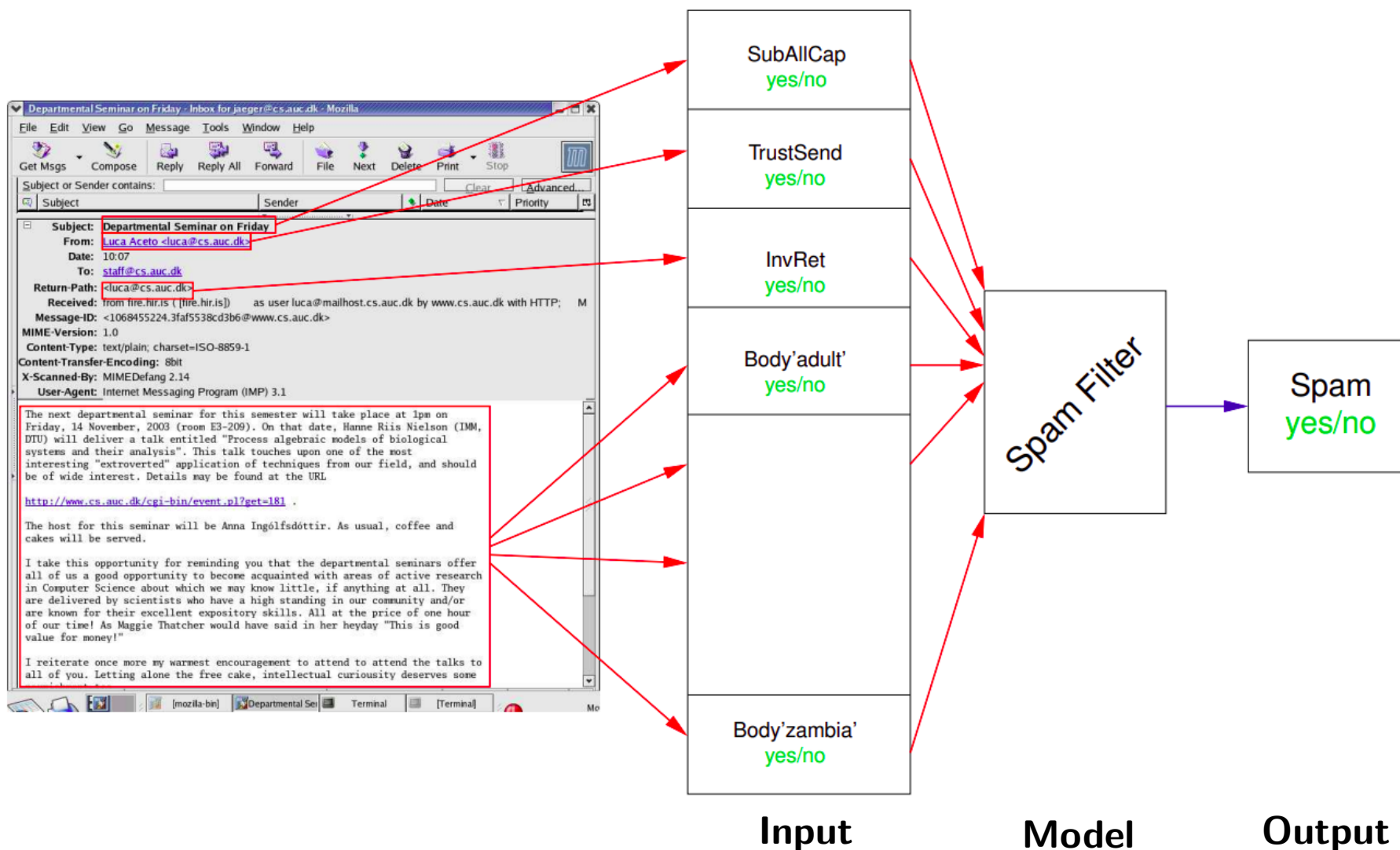
→ **We “supervise” the algorithm providing a set of examples!**

Labeled data: **Examples** or **Cases** of inputs that are **labeled** with the correct decisions (outputs).

Supervised Learning: Learn from a labeled dataset.

→ Learning is supervised because we are providing the correct answer (label) for the set of training examples

Example: Spam filter



Labeled Dataset

We assume that data (experience) consists of an **attribute-value table**:

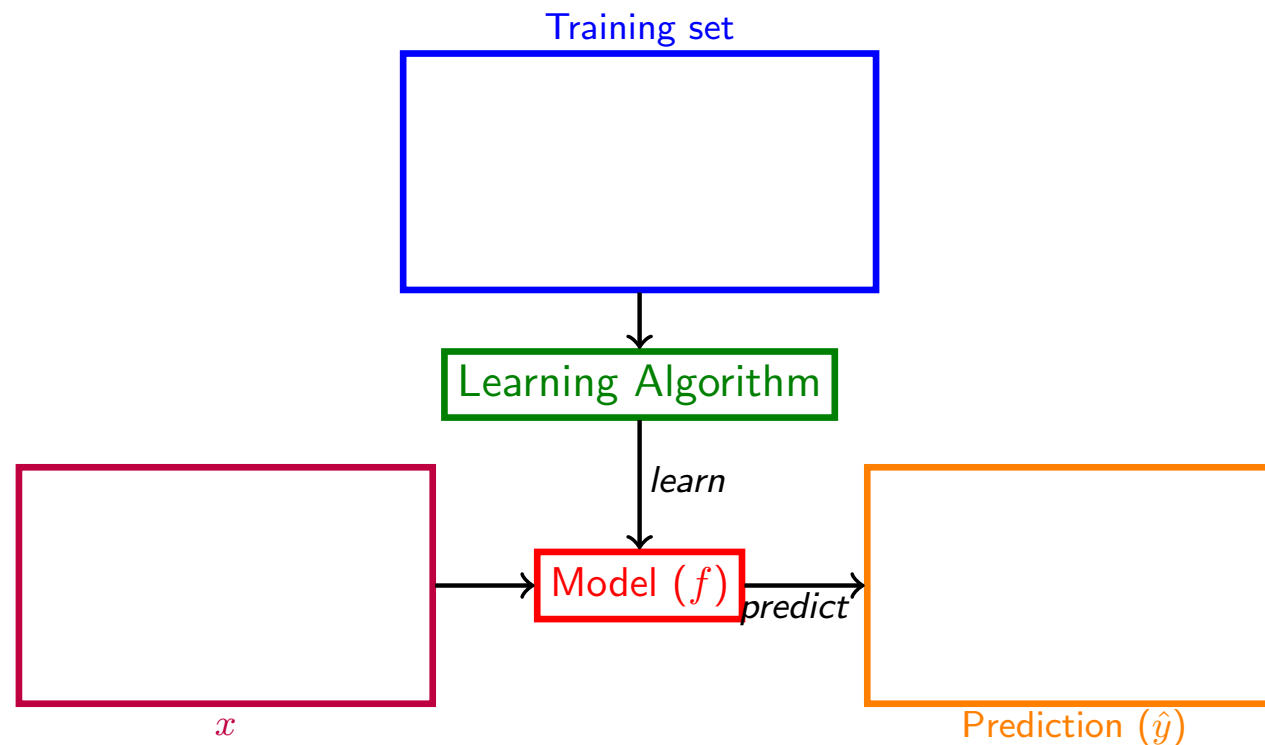
Input Features (Attributes, Predictor Variables)					Target Feature (Class variable/Label)
SubAllCap	TrustSend	InvRet	...	B'zambia'	Spam
y	n	n	...	n	y
n	n	n	...	n	n
n	y	n	...	n	y
n	n	n	...	n	n
...	...	...	...	...	...

- Columns correspond to **attributes** (also called features)
- Rows correspond to **examples** (also called cases or instances): observations of joint occurrences of values of the attributes.
- In prediction problems, there is a distinguished **target attribute**. When the target attribute is discrete, it is usually called the **class variable**.

In table above: *Spam* is class variable for the prediction problem: predict whether a mail is spam, given characteristics of the mail.

Supervised Learning Method

- First step: Learning from training data
 - Input: **Training set**
 - Output: **Model (f)**
- Second Step: Predicting the target feature using the learned model (f)
 - Input: **New data (x)**
 - Output: **Prediction feature (y)**



- Each example is a pair $(x, f(x))$ for some unknown function
- The learning algorithm returns a function that approximates f for a collection of examples

Continuous Attributes

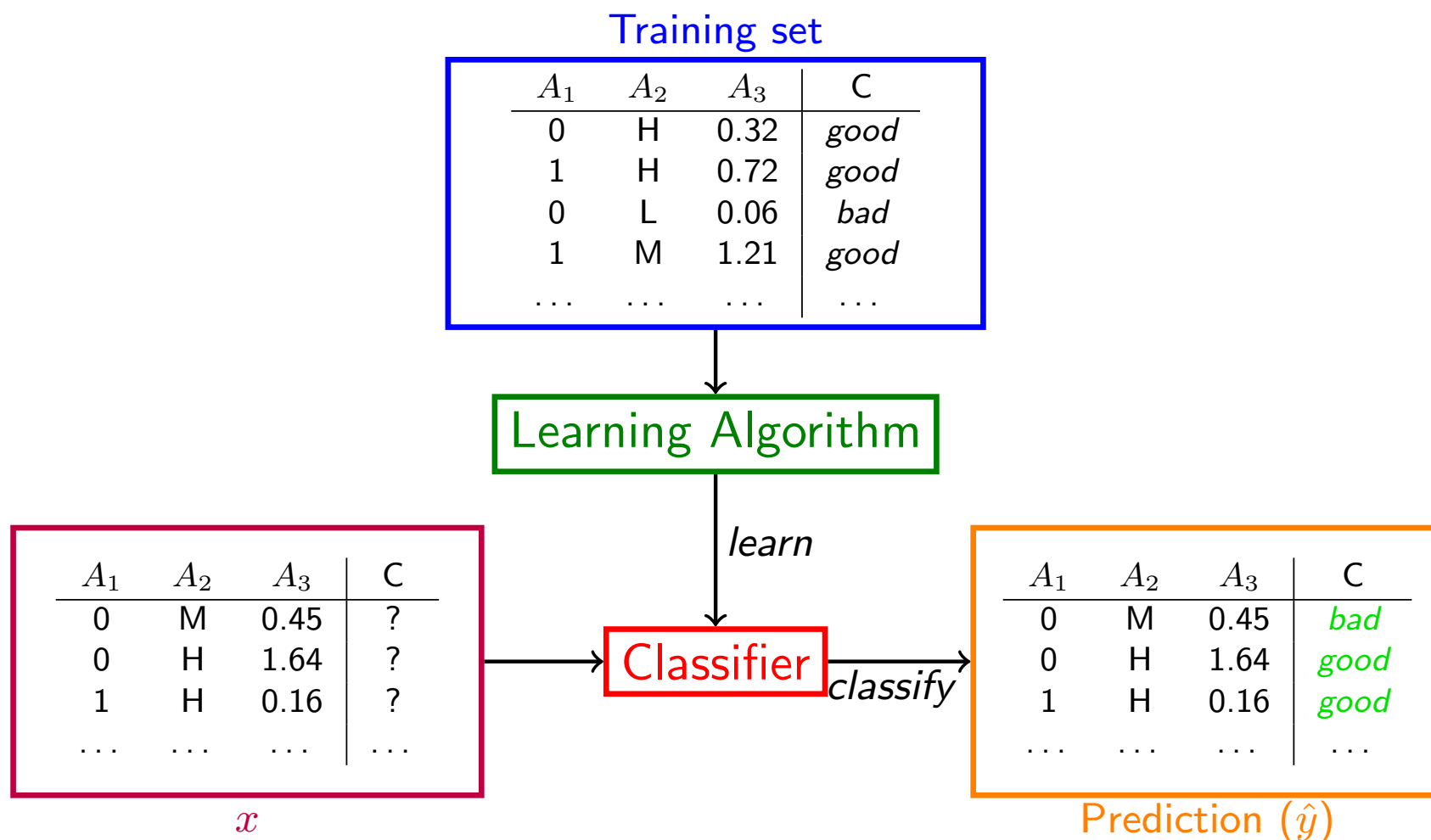
<i>current temp</i>	<i>pressure change (24h)</i>	<i>rain tomorrow</i>	<i>temp tomorrow</i>
16.8	-8.5	yes	15.3
21.7	2.1	no	22.5
19.5	-1.4	no	20.4
8.4	0.5	yes	7.2
...	...	...	...

~> one data set can be the basis for many different learning tasks:

- **Classification problem**: predict whether it rains tomorrow, given current temperature and pressure change (*rain tomorrow* is class variable).
- **Regression problem**: predict temperature tomorrow, given current temperature and pressure change (*temperature tomorrow* is target attribute).
- **Clustering problem** (**Chapter 8**): identify groups of observations representing similar weather patterns (e.g. stable winter high pressure situations).

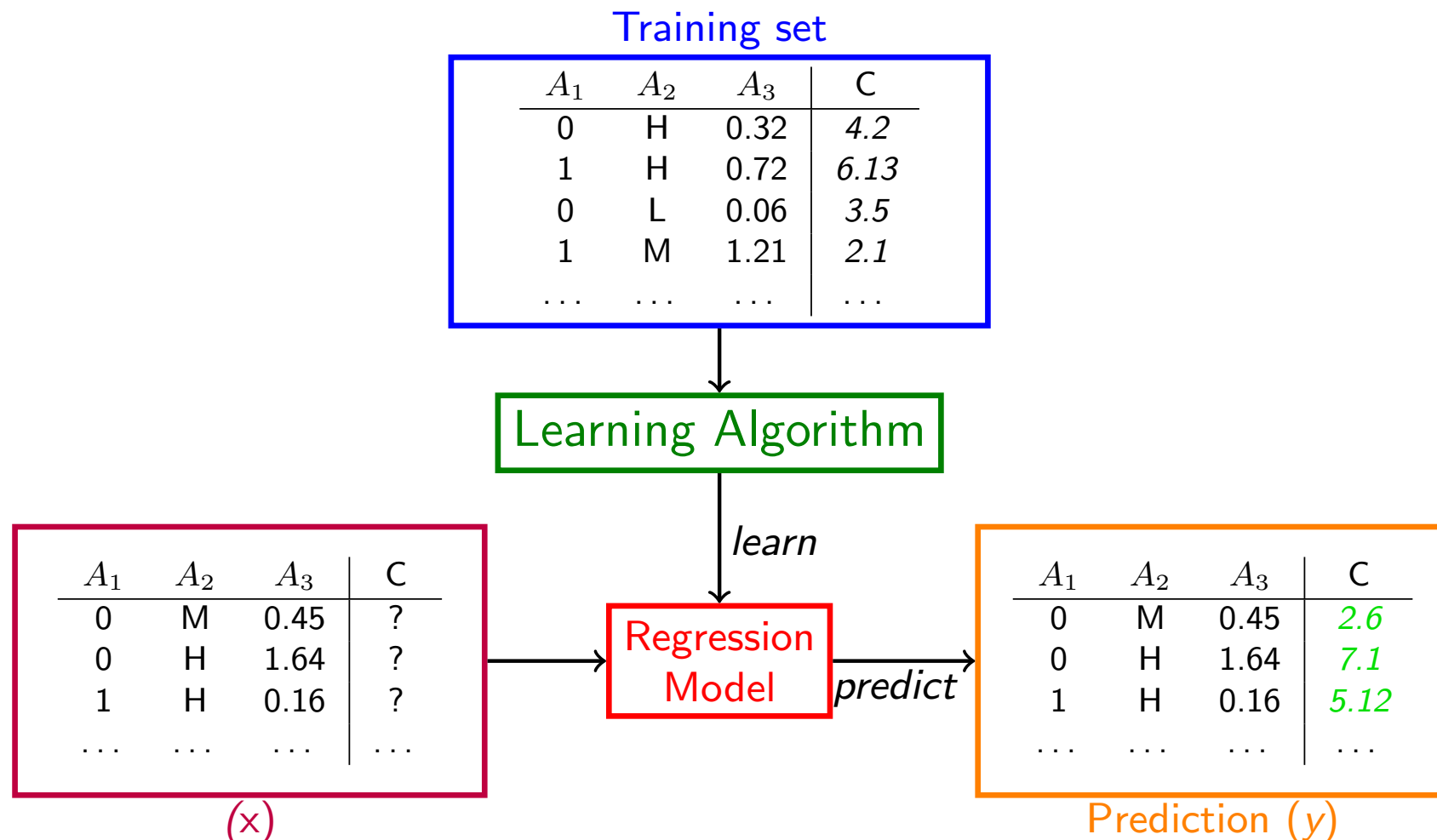
Classification Task

- Predicting a discrete target feature: **classification**
- Agent/Model/Program that classifies: **classifier**



Regression Task

- Predicting a continuous (numeric) target feature: **regression**
- Agent/Model/Program that performs regression: **regression model**



Ingredients

Key ingredients of a learning method are:

- a **Hypothesis Space**: set of all possible models that could be learned based on a given **Representation**
 - **Parameters**: Chosen by the learning algorithm (basically select a model from the hypothesis space)
 - **Hyperparameters**: Chosen by the user/hyperparameter optimization (restrict the hypothesis space)
- an **Evaluation Measure** that is used to decide how good a candidate hypothesis is. Typically, we try to **minimize an error function**: Given the decisions made by a model, what errors have they made on the training examples?

Learning Task: Find the model in the hypothesis space that minimizes the error (Note: This is not precise, as we will see when we come back to this in **Chapter 7**).

→ a **Search** or **Optimization** method used to find a hypothesis that scores high according to the evaluation measure

Regression

- Given a dataset, predict a continuous variable.
- Input variables $x_1, \dots, x_n$. Let's assume that they are numeric variables.

Example

Given data from a set of people (e.g. their weight), predict their height

Learn a function $\hat{y} = f(X)$ where y is the target value and $\hat{y}$ is the estimated value.

We denote Y to the vector of target values (for each example) and $\hat{Y}$ to the vector of estimated values

Error Functions for Regression

We can define multiple **error functions**, which compare vectors Y and $\hat{Y}$:

- **0/1 error** (L_0 norm)

$$\sum_{e \in E} (Y(e) \neq \hat{Y}(e))$$

→ **Minimize number of examples with error (disregard the amount of error)**

- **Sum Absolute Error (SAE)** (L_1 norm, Manhattan distance, mean absolute error (MAE):

$$\sum_{e \in E} |Y(e) - \hat{Y}(e)|$$

→ **Minimize the sum of errors**

- **Sum Squared Error (SSE)** (L_2 norm/Euclidean distance/mean squared error):

$$\sum_{e \in E} (Y(e) - \hat{Y}(e))^2$$

→ **Larger errors are more important than smaller errors** (having an error of 2 in an example is worse than having an error of 1 in two examples)

- **Worst-case error** (L_∞ norm):

$$\max_{e \in E} |Y(e) - \hat{Y}(e)|$$

→ **Minimize the worst-case error**

→ What error function is suitable may depend on the application and the learning method

Linear Regression

- Linear Regression assumes that the target attribute can be expressed as a linear combination of the input attributes:

$$\hat{y} = \sum_{i=1}^n w_i x_i + w_0$$

- Hypothesis space:** Hyperplanes in the $n + 1$ -dimensional space (if we have a single attribute x , this is just a line ($\hat{y} = mx + b$))
→ **Infinitely-many possible models!**

- Parameters:** select what model from the hypothesis space corresponds to our data

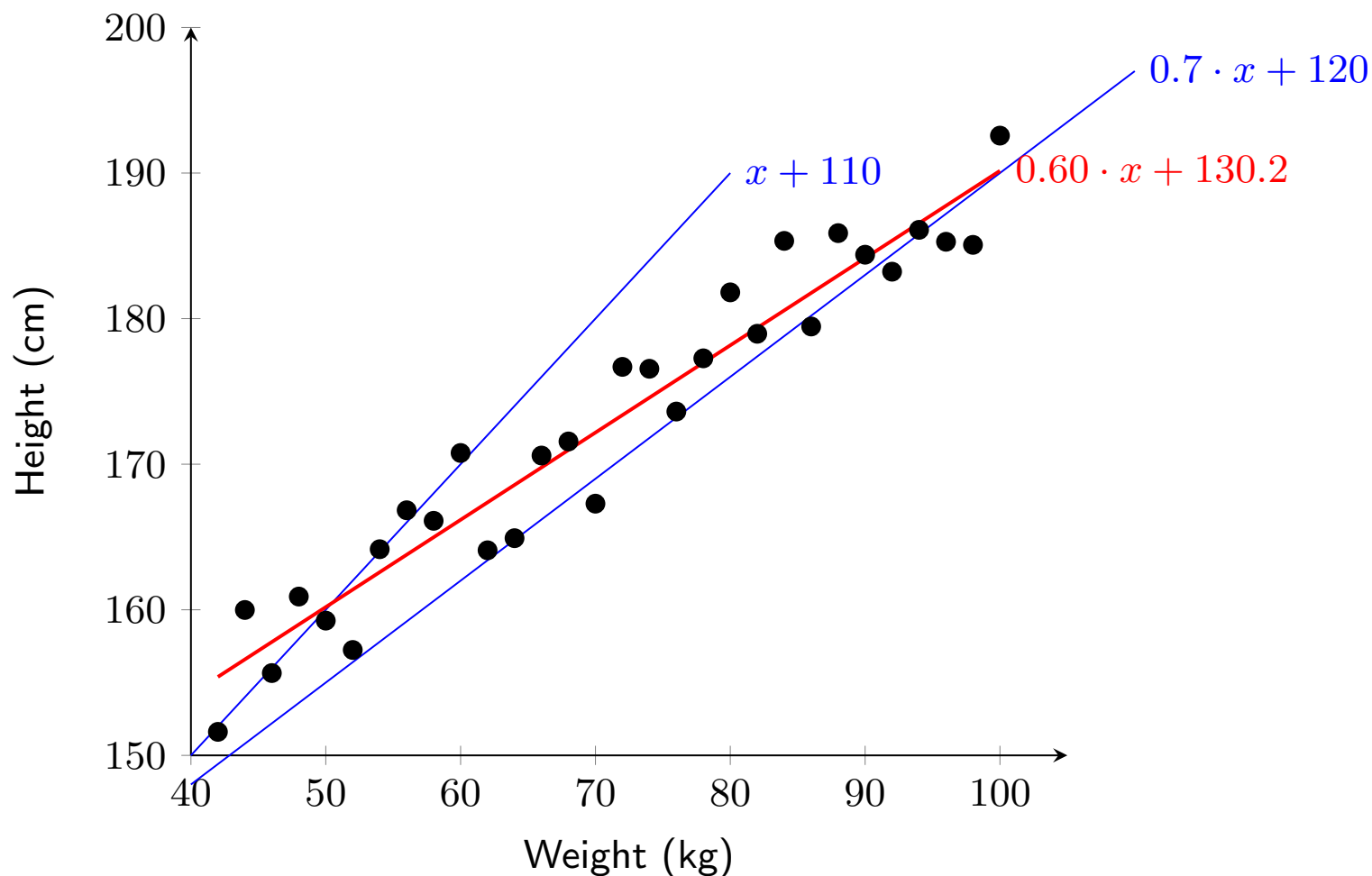
$$W = w_0, \dots, w_n$$

- A common error function is the **sum squared error (SSE)**:

$$SSE(Y, \hat{Y}) = \sum_{e \in E} (Y(e) - \hat{Y}(e))^2$$

Example: Linear Regression

Example: Input attribute $x = \text{Weight (kg)}$, Output attribute $y = \text{Height(cm)}$

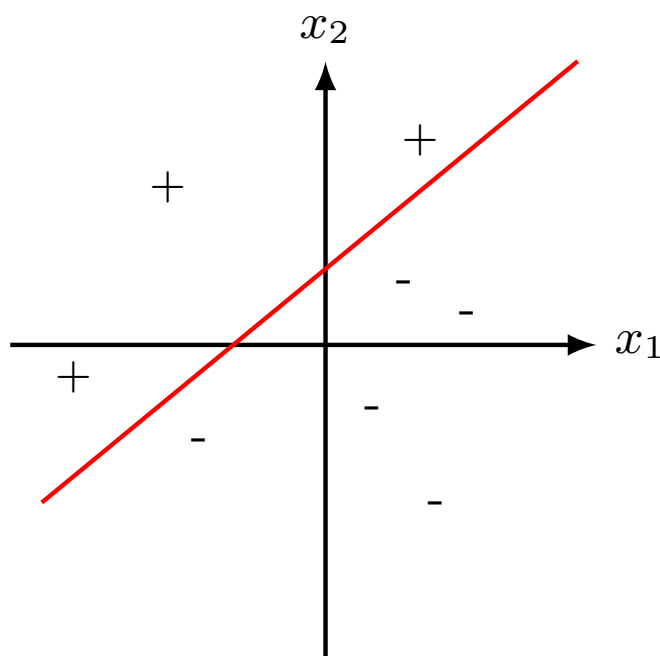


→ The red line is the one that minimizes the error with respect to the data (We will see how to find it in **Chapter 6**)

Side Note: Linear Classifier

We can use similar principles for classifications: e.g. if we can separate both classes with an hyperplane (a line if we have two dimensions)

Example: Input attributes x_1, x_2 , target attribute: (positive or negative)



Decision Trees

Definition (Decision Tree). *A tree in which:*

- *each internal (non-leaf) node is labeled with an attribute and/or condition, splitting the examples into 2 or more groups*
- *leaf nodes correspond to an estimate on the target class (most likely value or a probability estimate for each value)*

Example: Book Recommendation System

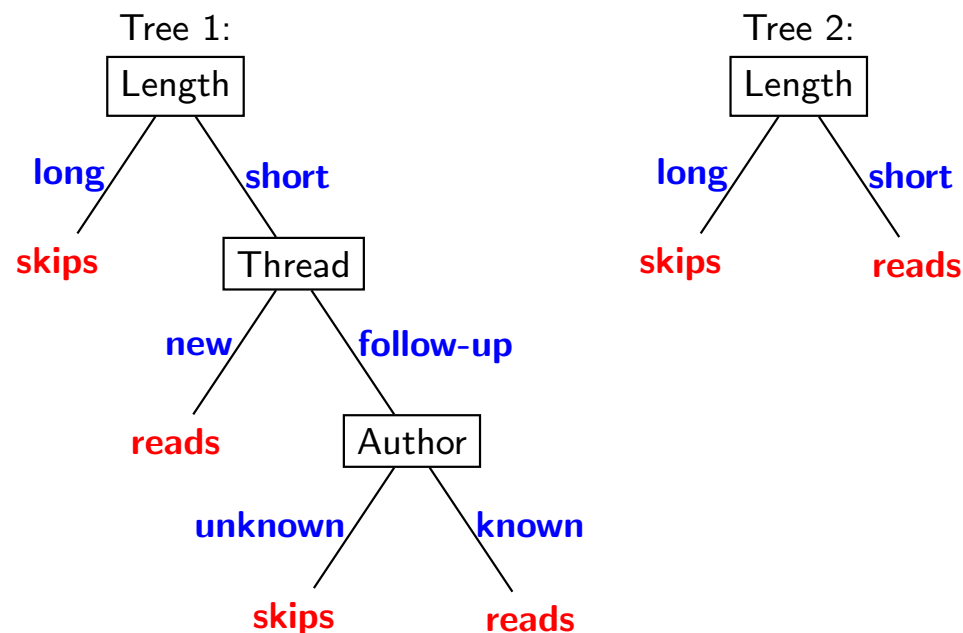
Training Data: we have collected data regarding the user preferences:
did they read the book or skip it?

Example	Author	Thread	Length	WhereRead	UserAction
e_1	known	new	long	home	skips
e_2	unknown	new	short	work	reads
e_3	unknown	follow up	long	work	skips
e_4	known	follow up	long	home	skips
e_5	known	new	short	home	reads
e_6	known	follow up	long	work	skips
e_7	unknown	follow up	short	work	skips
e_8	unknown	new	short	work	reads
e_9	known	follow up	long	home	skips
e_{10}	known	new	long	work	skips
e_{11}	unknown	follow up	short	home	skips
e_{12}	known	new	long	work	skips
e_{13}	known	follow up	short	home	reads
e_{14}	known	new	short	work	reads
e_{15}	known	new	short	home	reads
e_{16}	known	follow up	short	work	reads
e_{17}	known	new	short	home	reads
e_{18}	unknown	new	short	work	reads

We want to predict whether users will read/skip a book if we recommend it to them:

Example	Author	Thread	Length	WhereRead	UserAction
e_{19}	unknown	new	long	work	?
e_{20}	unknown	follow up	long	home	?
e_{21}	unknown	follow up	short	home	?

Example: Decision Trees for Book Recommendation



Side note: Decision trees can be interpreted as programs with limited syntax: logic programs without variables and/or imperative programs with only if/else statements

Tree 1 is equivalent to the following programs:

$skips \leftarrow long$
 $reads \leftarrow short \wedge new$
 $reads \leftarrow short \wedge follow_up \wedge known$
 $skips \leftarrow short \wedge follow_up \wedge unknown$

```

1  if Length = short then
2    |   if Thread = follow-up then
3        |       if Author = known then
4            |       return reads;
5        |       else
6            |       return skips;
7        |   else
8            |   return reads ;
9  else
10 |   return skips;
  
```

Decision Trees

Hypothesis Space? The set of all possible decision trees that we could create

Question!

What is the size of the hypothesis space of trees if we have 4 binary features and a binary class?

(A): 16

(B): 24

(C): 384

(D): 65536

65536. We have 2^n potential examples and for each of them we can classify them in two classes.

This is double exponential: 2^{2^n} .

With 6 attributes, we have 18 446 744 073 709 551 616 trees, and with 10 attributes $\approx 10^{308}$.

Example: Reading recommendation: Classifications

Example	Author	Thread	Length	WhereRead	UserAction	Tree 1	Tree 2
e_1	known	new	long	home	skips	skips	skips
e_2	unknown	new	short	work	reads	reads	reads
e_3	unknown	follow up	long	work	skips	skips	skips
e_4	known	follow up	long	home	skips	skips	skips
e_5	known	new	short	home	reads	reads	reads
e_6	known	follow up	long	work	skips	skips	skips
e_7	unknown	follow up	short	work	skips	skips	reads
e_8	unknown	new	short	work	reads	reads	reads
e_9	known	follow up	long	home	skips	skips	skips
e_{10}	known	new	long	work	skips	skips	skips
e_{11}	unknown	follow up	short	home	skips	skips	reads
e_{12}	known	new	long	work	skips	skips	skips
e_{13}	known	follow up	short	home	reads	reads	reads
e_{14}	known	new	short	work	reads	reads	reads
e_{15}	known	new	short	home	reads	reads	reads
e_{16}	known	follow up	short	work	reads	reads	reads
e_{17}	known	new	short	home	reads	reads	reads
e_{18}	unknown	new	short	work	reads	reads	reads
e_{19}	unknown	new	long	work	?	skips	skips
e_{20}	unknown	follow up	long	home	?	skips	skips
e_{21}	unknown	follow up	short	home	?	skips	reads

Learning Decision Trees: ID3 Algorithm

Top-down construction:

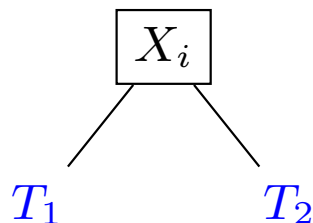
procedure *DecisionTreeLearner*($\mathbf{X}, Y, E$)

// $\mathbf{X} = \{X_1, \dots, X_n\}$: input features

// Y : target feature

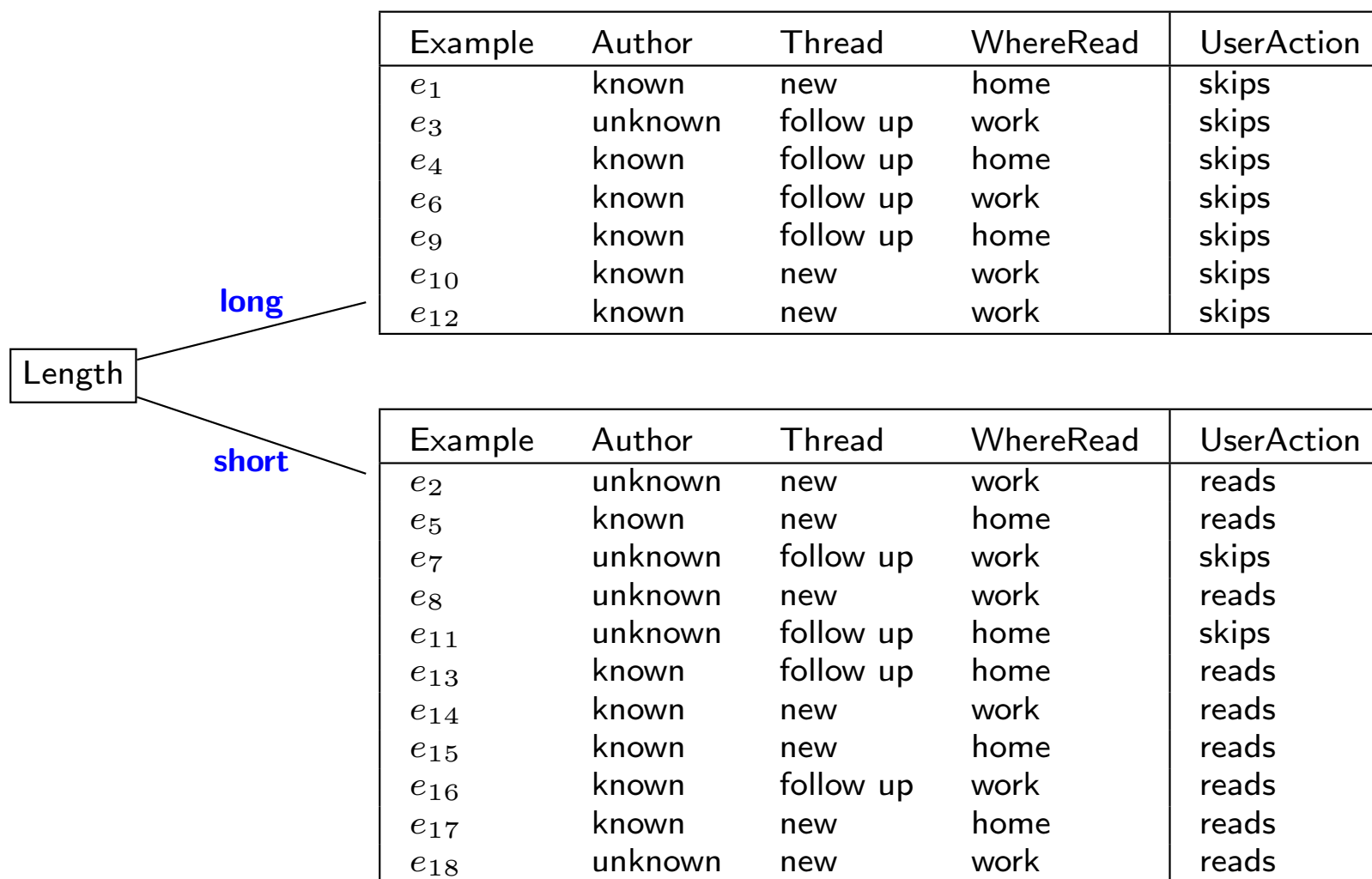
// E : set of training examples

1. **if** stopping criterion is true
2. **return** *leaf node* labeled with most frequent target feature value in E
3. **else**
4. select feature $X_i \in \mathbf{X}$
 // let v_1, v_2 be the possible values of X_i
5. $E_1 = \{e \in E : \text{val}(e, X_i) = v_1\}$
6. $T_1 = \text{DecisionTreeLearner}(\mathbf{X}, Y, E_1)$
7. $E_2 = \{e \in E : \text{val}(e, X_i) = v_2\}$
8. $T_2 = \text{DecisionTreeLearner}(\mathbf{X}, Y, E_2)$
9. **return**



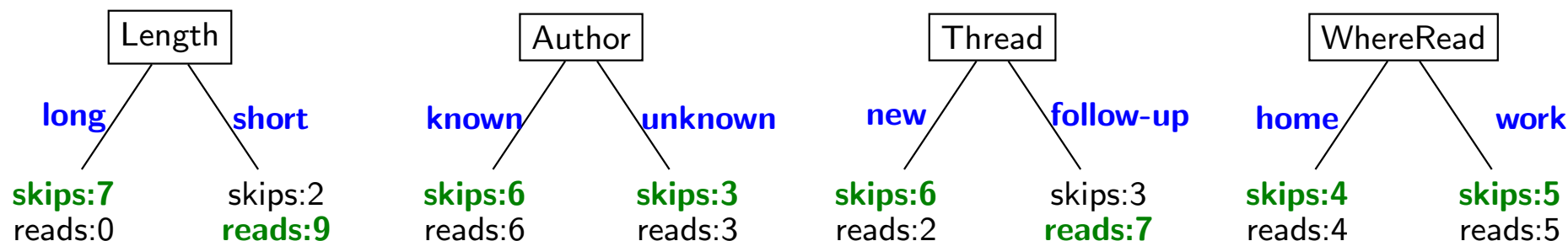
Example: Book Recommendation System

We choose an attribute and split our data according to its value:



Choosing X_i

Number of examples with class labels skips,reads belonging to different sub-trees:



Question!

Which attribute X_i should we use in line 4. to split the data in the root node?
Why?

(A): Length

(B): Author

(C): Thread

(D): WhereRead

- Possible approach: choose the feature that would provide the best classifier if construction would terminate with that feature.
- Next we see an even better criterion: maximize **information gain** measured in terms of entropy

Class Purity

Principle: Prefer features that split the examples into *class pure* subsets:

pure:

skips: 7, reads: 0
skips: 0, reads: 5
skips: 11, reads: 0

nearly pure:

skips: 6, reads: 1
skips: 2, reads: 15
skips: 11, reads: 1

impure:

skips: 6, reads: 5
skips: 7, reads: 7
skips: 13, reads: 12

Normalized to probabilities:

pure:

skips: 1, reads: 0
skips: 0, reads: 1
skips: 1, reads: 0

nearly pure:

skips: 0.85, reads: 0.15
skips: 0.12, reads: 0.88
skips: 0.91, reads: 0.09

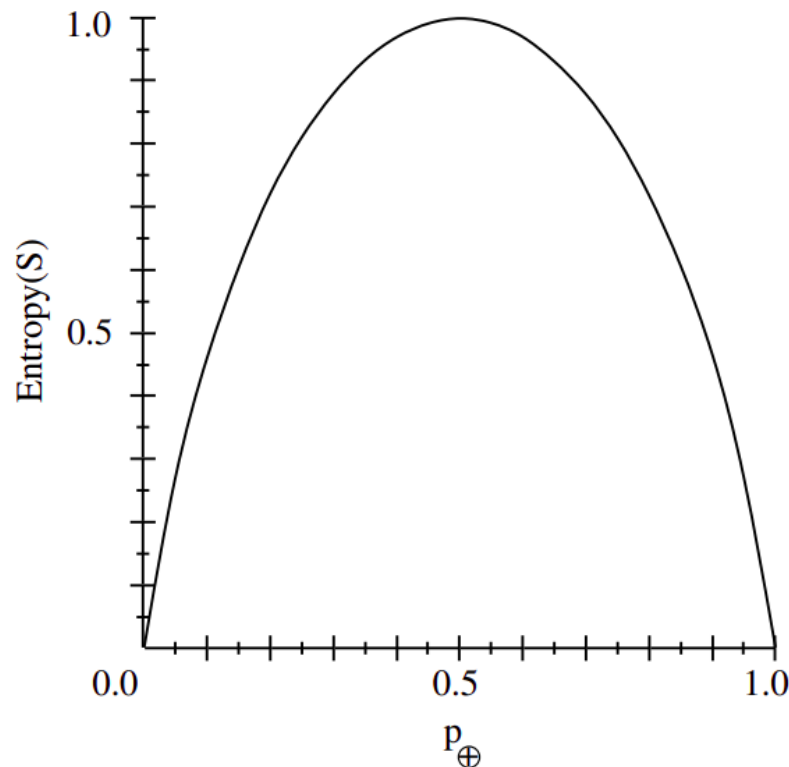
impure:

skips: 0.54, reads: 0.46
skips: 0.5, reads: 0.5
skips: 0.52, reads: 0.48

Entropy for Binary Classes

For a probability distribution $(p, 1 - p)$ of a two-valued class label, define

$$\text{Entropy}(p, 1 - p) = -p \cdot \log_2(p) - (1 - p) \cdot \log_2(1 - p)$$



- High values for impure distributions
- Maximal for $(0.5, 0.5)$
- Zero for $(1, 0)$ and $(0, 1)$

Examples:

- $\text{Entropy}(0.5, 0.5) = -0.5 \cdot \log_2(0.5) - 0.5 \cdot \log_2(0.5) = 0.5 \cdot 1 + 0.5 \cdot 1 = 1$
- $\text{Entropy}(0.35, 0.65) = -0.35 \cdot \log_2(0.35) - 0.65 \cdot \log_2(0.65) = 0.93$
- $\text{Entropy}(0, 1) = -0 \cdot \log_2(0) - 1 \cdot \log_2(1) = -0 - 0 = 0$
- $\text{Entropy}(1, 0) = -1 \cdot \log_2(1) - 0 \cdot \log_2(0) = -0 - 0 = 0$

Entropy for Multi-valued Attributes

For a probability distribution on domain with n elements:

$$\mathbf{p} = (p_1, p_2, \dots, p_n) \quad \left(\sum_{i=1}^n p_i = 1 \right)$$

define entropy:

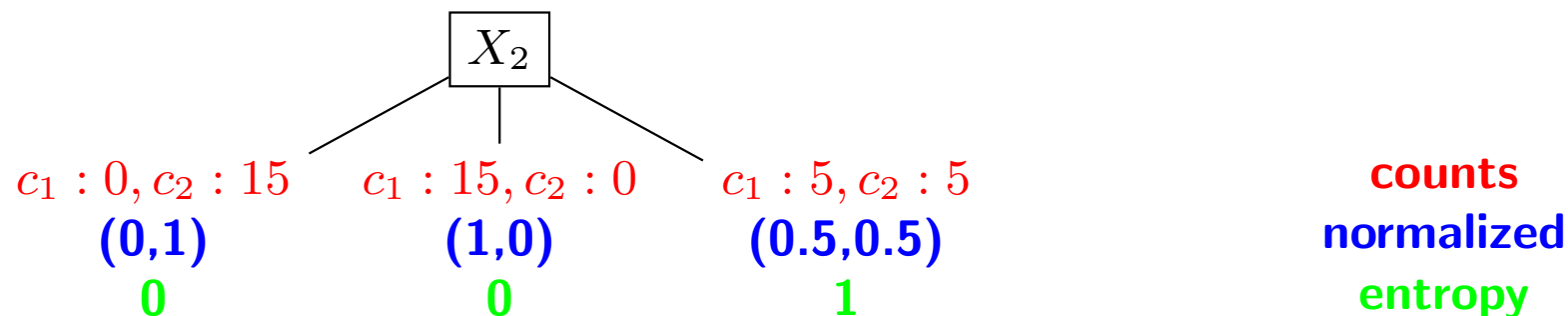
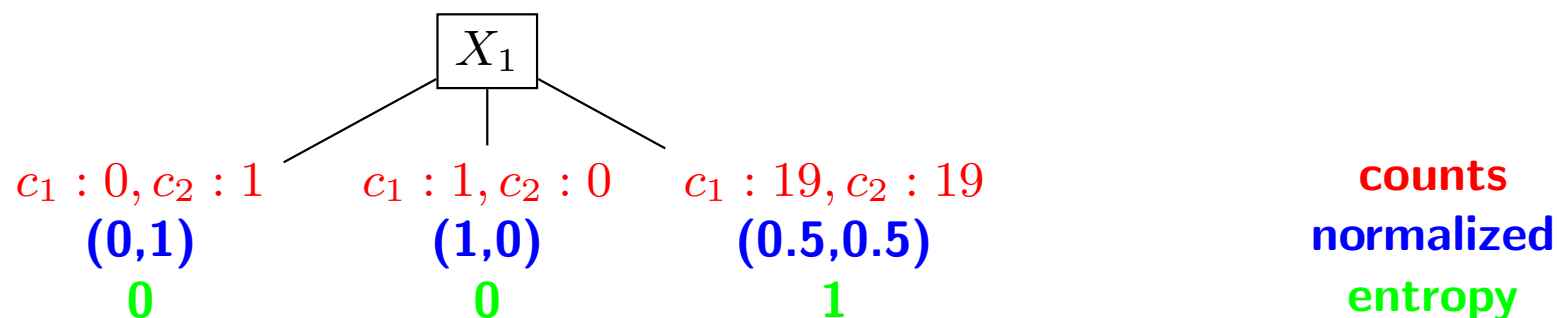
$$Entropy(\mathbf{p}) = - \sum_{i=1}^n p_i \cdot \log_2(p_i)$$

Again:

- Maximal for $\mathbf{p} = (1/n, \dots, 1/n)$
- Zero for $\mathbf{p} = (1, 0, \dots, 0)$, $\mathbf{p} = (0, 1, 0, \dots, 0), \dots, \mathbf{p} = (0, \dots, 0, 1)$

Expected Entropy Example

We prefer features that split into subsets with low entropy for the target class ($\text{Entropy}(\text{Class} \mid X)$), but we need to consider also the amount of examples in each of the branches. Consider the following example where we choose to split the data according to feature X_1 or X_2 :



X_2 provides a better division of examples than X_1 . It gives a lower *expected entropy*:

$$(1/40) \cdot 0 + (1/40) \cdot 0 + (38/40) \cdot 1 > (15/40) \cdot 0 + (15/40) \cdot 0 + (10/40) \cdot 1$$

Expected Entropy and Information Gain

Definition (Expected Entropy). For feature X with domain $v_1, \dots, v_n$, let:

- E_i be the set of examples with $X = v_i$
- $q_i = |E_i|/|E|$
- h_i the entropy of the class label distribution in E_i

The **expected entropy** from splitting on X is:

$$Entropy(Class \mid X) = \sum_{i=1}^n q_i \cdot h_i$$

Definition (Information Gain).

Let $Entropy(Class)$ be the entropy of class label distribution before splitting. The **Information Gain** from splitting on X then is:

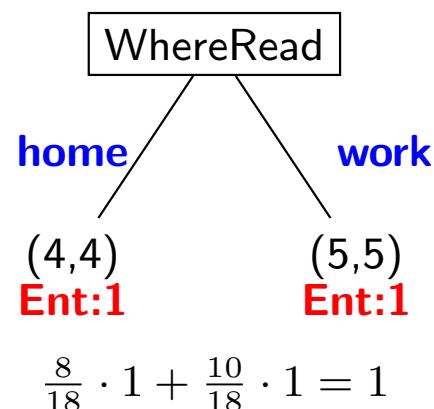
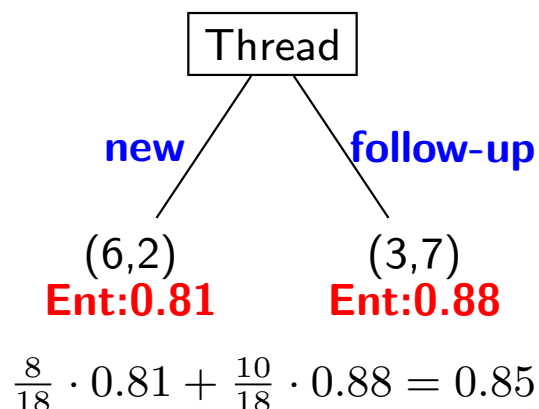
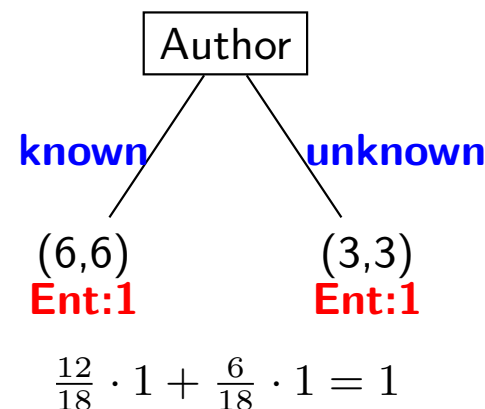
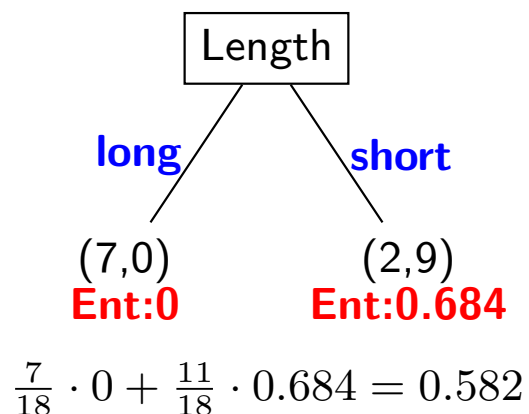
$$Entropy(Class) - Entropy(Class \mid X)$$

Information Gain in Decision Tree Learning

→ In line 4. of algorithm choose feature X_i that gives highest information gain.

Example: Information Gain on Book Recommendation Classifier

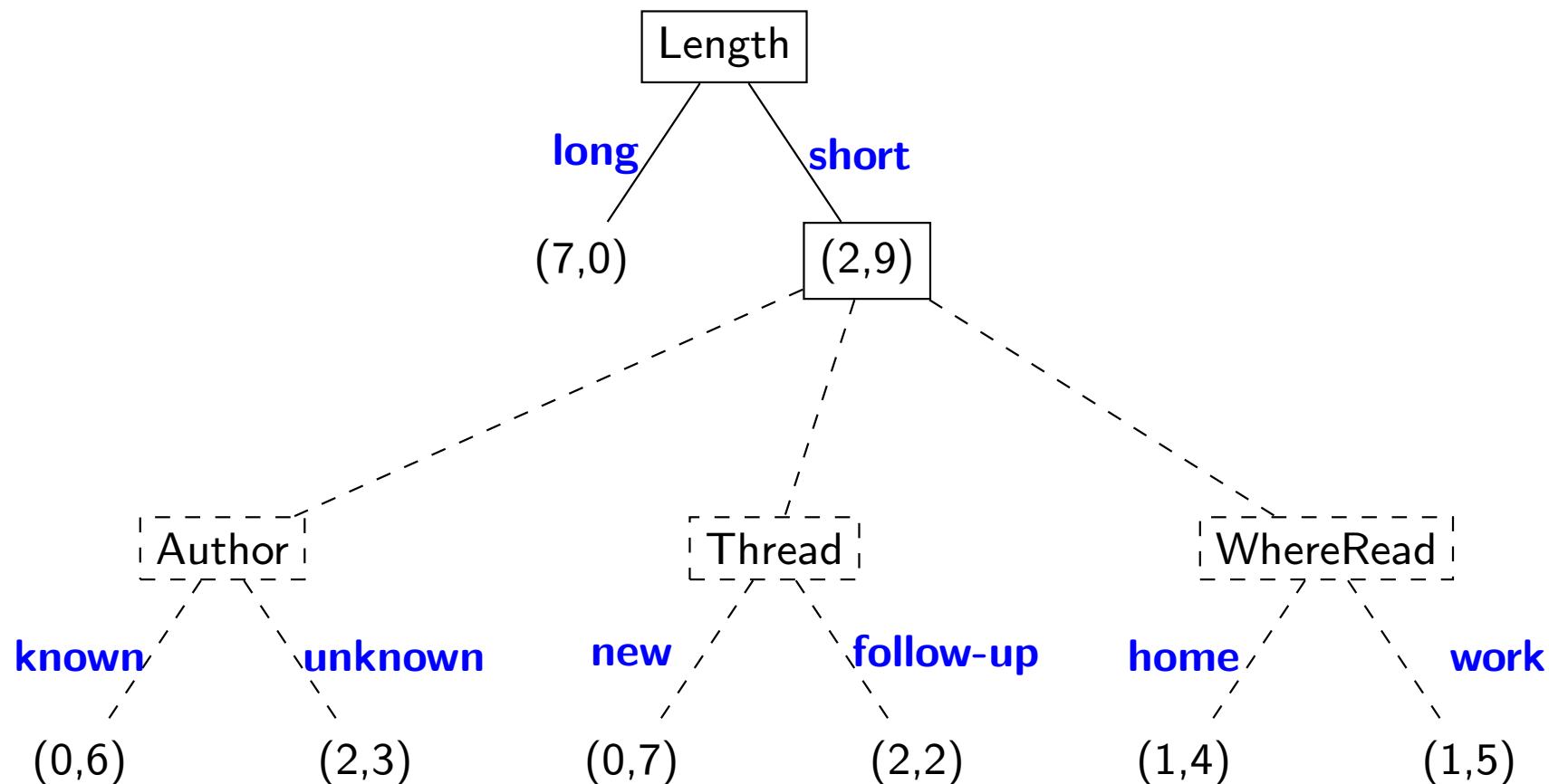
Entropy at the root: $(9, 9) \rightarrow 1$



→ Split on Length, as it has largest information gain $(1 - 0.582 = 0.418)$

Example: Constructing the Decision Tree for Book Recommendation

As we still have an impure node, we continue splitting the tree:



Stopping criteria:

- No split gives enough information gain (according to a threshold)
- Limits on the maximum depth of the tree

Dealing with Many-valued attributes

The **information gain** measure favors attributes with many values:

- For example, the attribute **Date** (with the possible dates as states) will have a very high **information gain** but is unable to generalize!

One approach for avoiding this problem is to select **attributes** (X) based on Gain Ratio (for a given set of examples S):

$$\text{GainRatio}(X) = \frac{\text{InformationGain}(X)}{\text{SplitInformation}(S, \mathbf{X})}$$

$$\text{SplitInformation}(S, \mathbf{X}) = - \sum_{i=1}^c \frac{|S_i|}{|S|} \log_2 \frac{|S_i|}{|S|},$$

where S_i is the subset of examples produced by splitting on the i 'th value of $\mathbf{X}$.

Note that **SplitInformation** is the entropy of S w.r.t. the values of $\mathbf{X}$. (so that we prefer splits

Continuous Attributes

We require that the attributes being tested are discrete valued. So in order to test a continuous valued attribute we need to “discretize” it.

Suppose that the training examples are associated with the attribute **Temperature**:

Temperature:	40	48	60	72	80	90
Rain tomorrow:	yes	yes	no	no	no	yes

Create a new boolean valued **attribute** by first testing the two candidate thresholds:

- $(48+60)/2$
- $(80+90)/2$

Next, pick the one with highest information gain (i.e., **Temperature**_{>54})

Summary

- **Supervised Learning** techniques learn from examples that have been labelled with the correct “answer”
- **Classification** is about predicting discrete variables. **Regression** is about predicting continuous attributes.
- The supervised learning task consists of finding the model that minimizes some error function over the set of training examples.
- Linear Regression assumes a simple structure $y = \sum w_i x_i + w_0$, and focuses on finding the parameters W that minimize the error
- We can learn decision trees for classification by recursively decomposing our data.
- A way to guide the search for good decision trees is to (greedily) maximize information gain (i.e. minimize entropy).

Reading

- *Chapter 7: Supervised Machine Learning* from the book "Artificial Intelligence: Foundations of Computational Agents (2nd edition)"
In particular:
 - *Chapter 7.1: Learning Issues*
 - *Chapter 7.2: Supervised Learning*
 - *Chapter 7.3: Basic Models for Supervised Learning*
 - *Chapter 7.3.2: Linear Regression and Classification*
 - *Chapter 7.3.1: Learning Decision Trees*